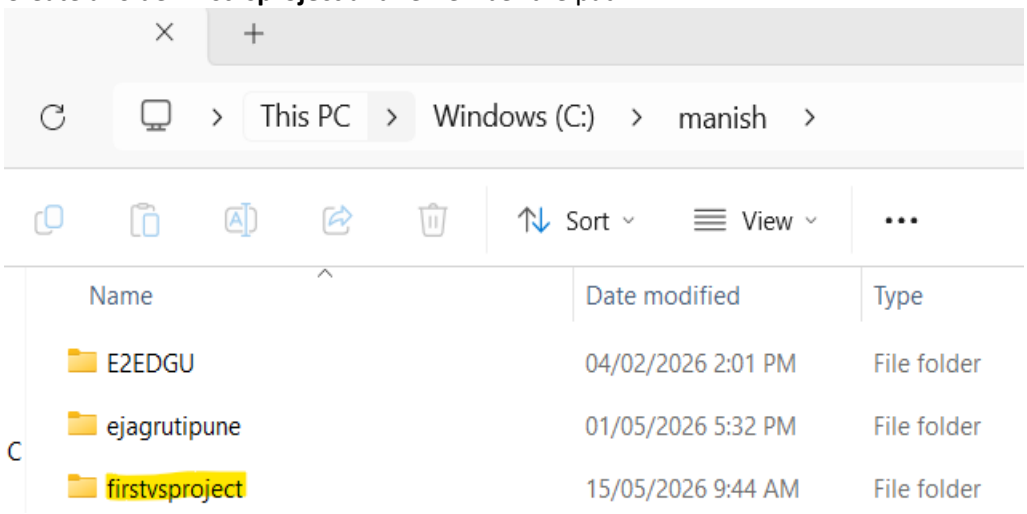
1. Download visual studio code from https://code.visualstudio.com/Download
2. Refer uv-Astral Docs https://docs.astral.sh/uv
3. Open powershell terminal in visual studio code and run below command to install **uv**
   powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
4. uv is a **modern Python package manager + environment manager** designed to

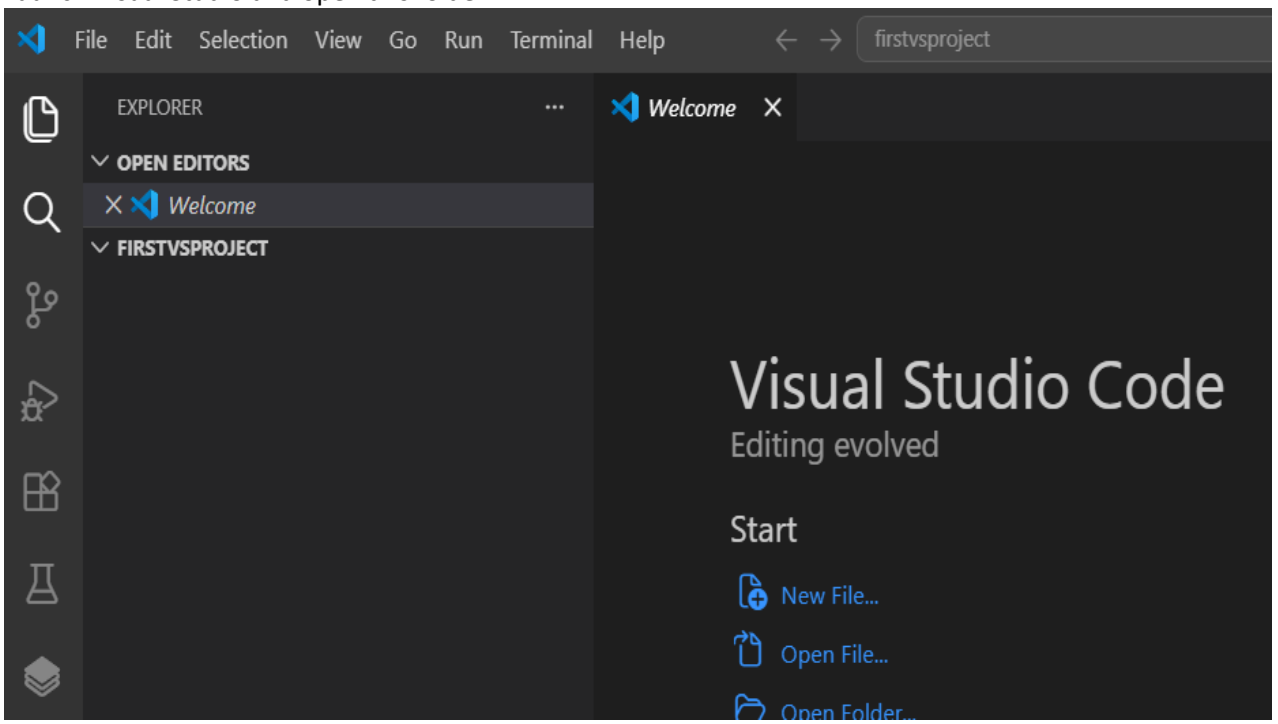
Aspect	**Traditional (pip + venv)**	**uv (modern tool)**
Purpose	Separate tools for env + packages	One unified tool
Environment creation	python -m venv venv	uv venv
Package install	pip install package	uv pip install package
Speed	Slower (Python-based)	Very fast (Rust-based) ⚡
Dependency management	Basic (requirements.txt)	Better resolution (like modern tools)
Disk usage	High (duplicate packages per env)	Low (shared cache) 💾
Package storage	Inside each venv	Global cache + linked to env
Reusability	Low (each env independent copy)	High (shared packages across projects)
Setup complexity	Multiple tools needed	Single tool ✅
Isolation	Yes (via venv)	Yes (via uv venv)
Best for	Small / simple projects	Large / multiple / ML projects
Performance	Standard	Optimized (parallel, cached) 🚀

5. Make sure that **uv** is properly installed and installation path set to windows env variable
   $env:PATH += ";C:\Users\ADMIN\.local\bin"

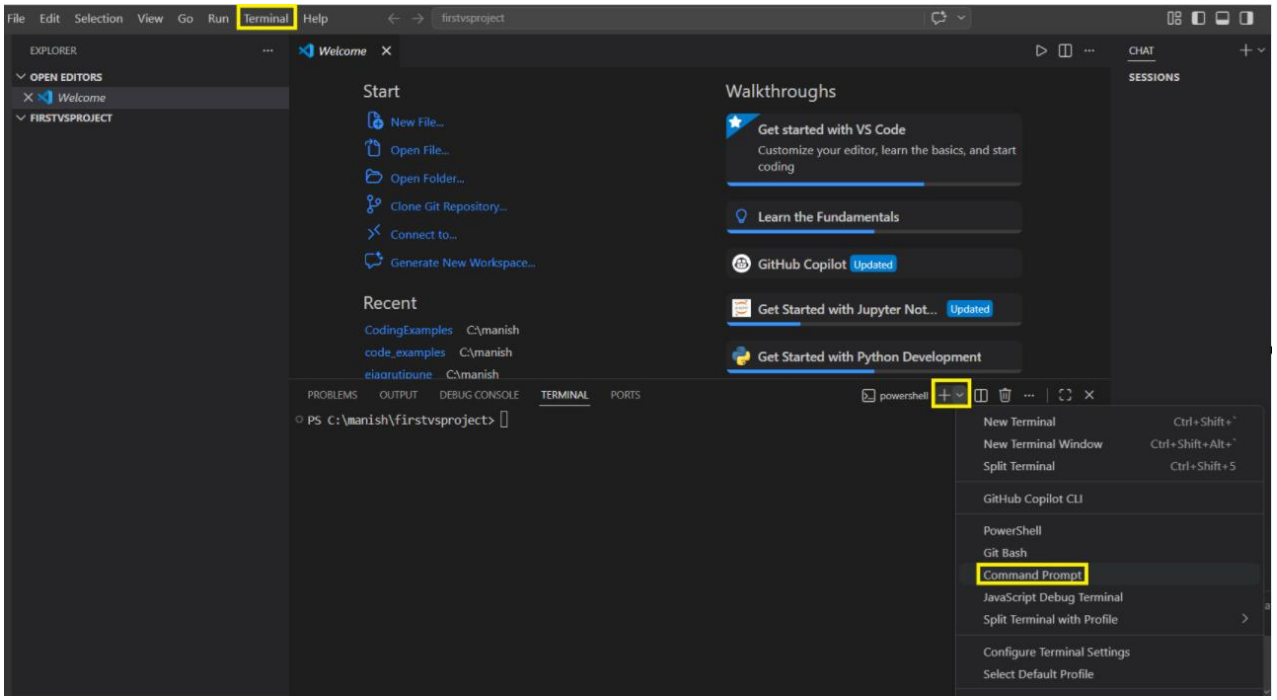
6. Create a folder **firstvsproject** and remember the path



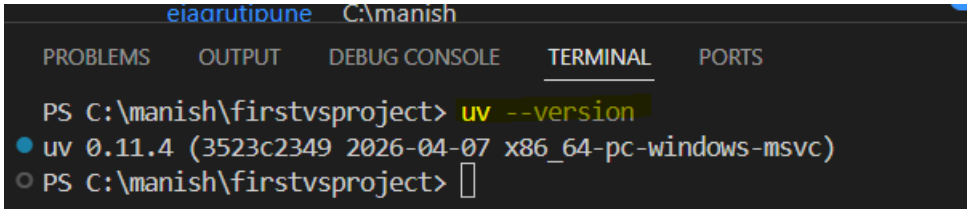
7. Launch Visual Studio and open this folder



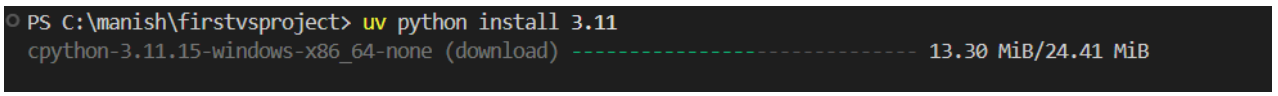
8. Open Command Prompt Terminal in Visual Studio Code

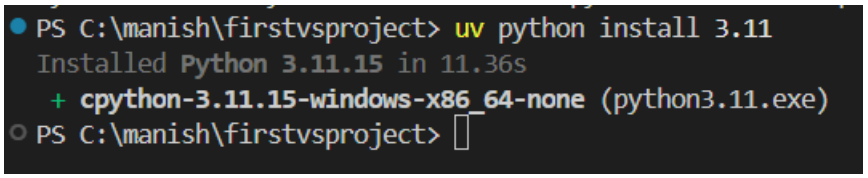


9. Type the command **uv –version** to make sure **uv** is properly installed



10. Now install python using uv, run below command in command line **uv python install 3.11**





Note: uv help us to run our code on different versions of python in the same machine



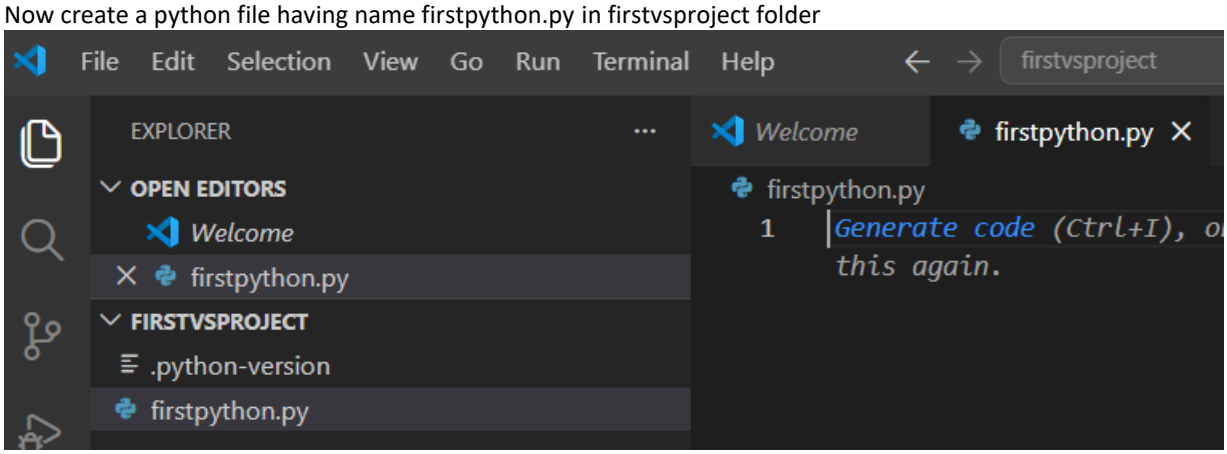
11. If you want your project to refer 3.13 then

```
PS C:\manish\firstvsproject> uv python pin 3.13
Pinned `.python-version` to `3.13`
PS C:\manish\firstvsproject>
```

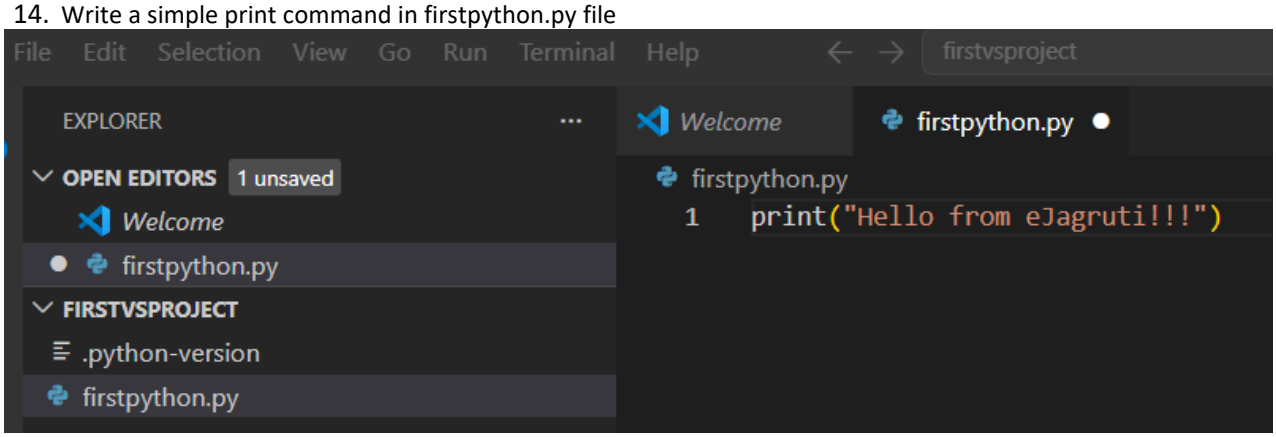
12. If you want your project to refer 3.12 then

```
PS C:\manish\firstvsproject> uv python pin 3.12
Updated `.python-version` from `3.13` -> `3.12`
PS C:\manish\firstvsproject>
```

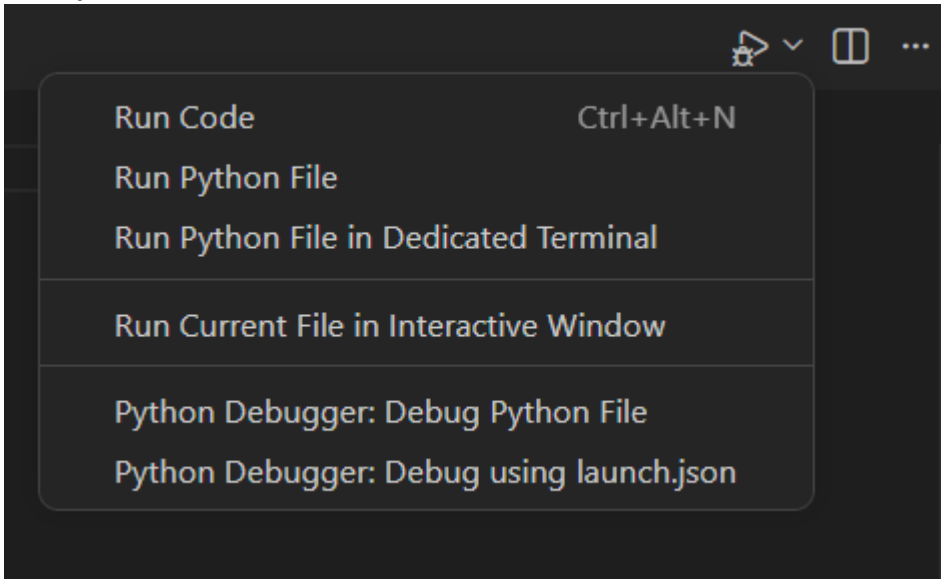
13. Now create a python file having name firstpython.py in firstvsproject folder



14. Write a simple print command in firstpython.py file



15. To Run the code click on the play button on the RHS and
Run Python File



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  ...   Filter (e.g. text, !excludeText, t...   Code
[Running] python -u "c:\manish\firstvsproject\firstpython.py"
Hello from eJagruti!!!

[Done] exited with code=0 in 0.273 seconds
```

16. How to add library?

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\manish\firstvsproject> uv pip install numpy
Checked 1 package in 3ms
PS C:\manish\firstvsproject> uv pip install pandas
Resolved 5 packages in 683ms
Prepared 1 package in 3.27s
Installed 4 packages in 1.21s
 + pandas==3.0.3
 + python-dateutil==2.9.0.post0
 + six==1.17.0
 + tzdata==2026.2
PS C:\manish\firstvsproject>
```

17. Above library will be downloaded below location

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\manish\firstvsproject> uv pip show numpy
Name: numpy
Version: 2.4.4
Location: C:\manish\firstvsproject\.venv\Lib\site-packages
Requires:
Required-by: pandas
PS C:\manish\firstvsproject>
```

Note: uv always required virtual environment to store the library if virtual environment is not set then you will get the below error

```
error: No virtual environment found; run `uv venv` to create an environment, or pass `--system`
to install into a non-virtual environment
```

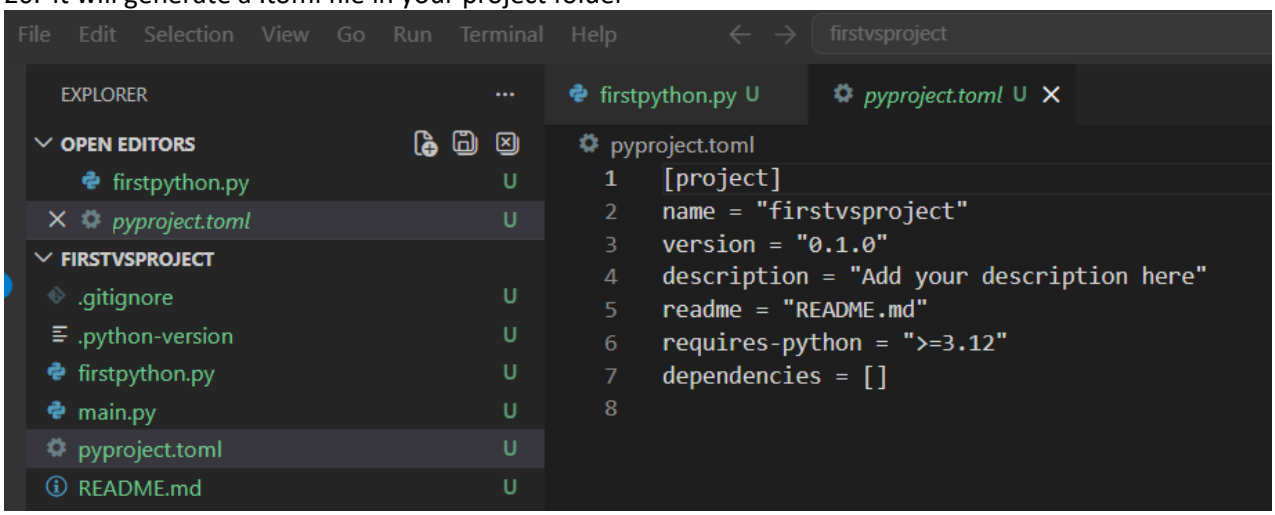
18. If you want a isolated environment for your project, then you need to create a virtual environment for your project. Follow given below steps
19. Open Terminal>>Command Prompt

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Microsoft Windows [Version 10.0.22631.6936]
(c) Microsoft Corporation. All rights reserved.

C:\manish\firstvsproject>uv init
Initialized project `firstvsproject`

C:\manish\firstvsproject>
```

20. It will generate a .toml file in your project folder



21. What is virtual environment and why we need it

A **virtual environment** is:

> A **separate isolated Python environment** with its own packages and dependencies.

Purpose

- Avoid dependency conflicts
- Keep project dependencies isolated
- Allow different projects to use different library versions

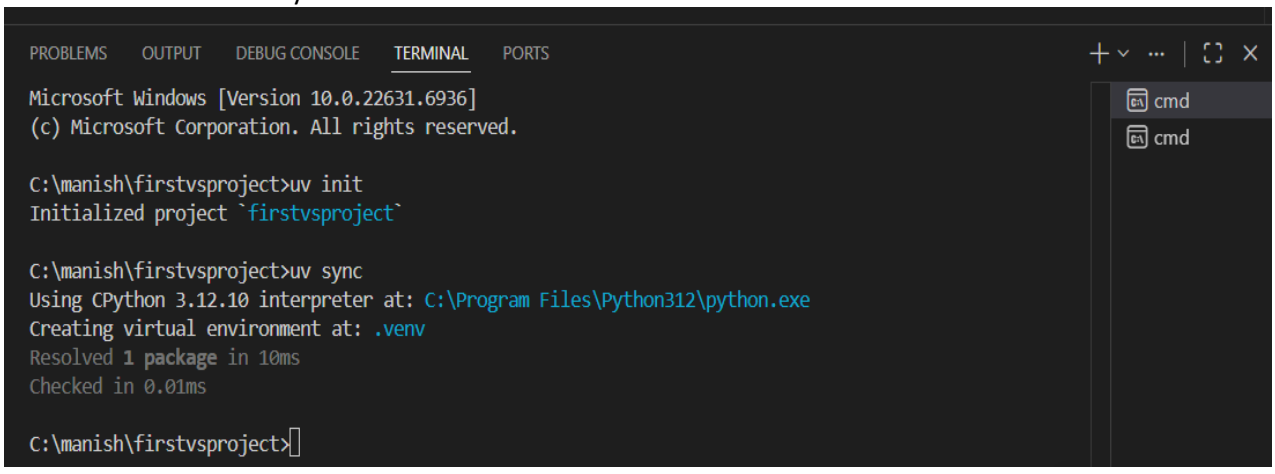
Example

You may have:

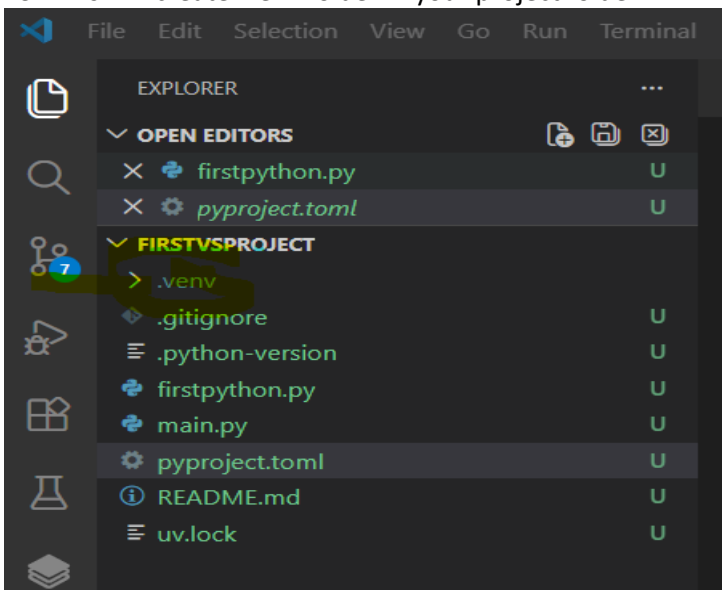
- Project A → needs `numpy==1.21`
- Project B → needs `numpy==1.26`

Virtual environments let both run without conflict.

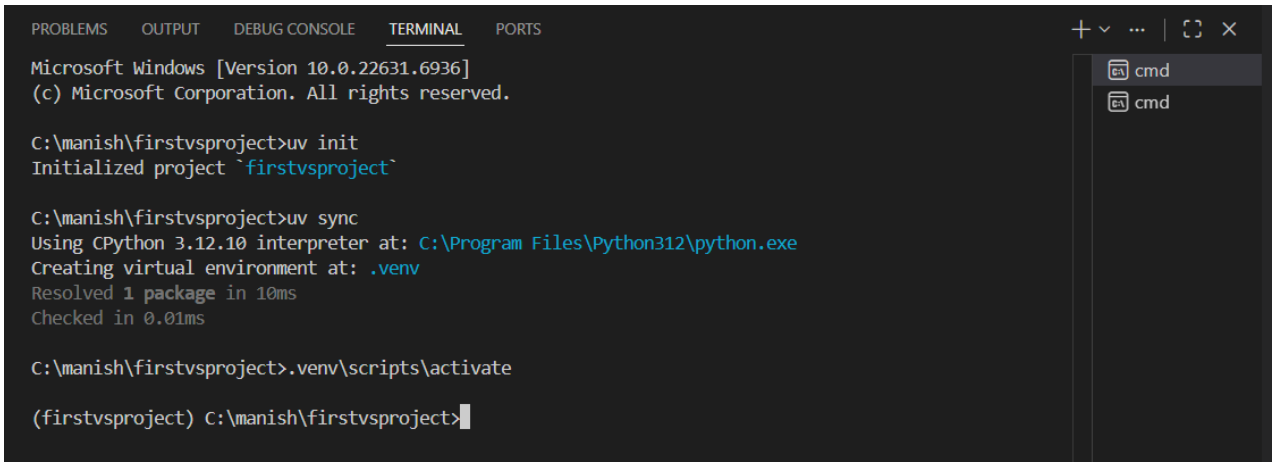
22. Run command : uv sync



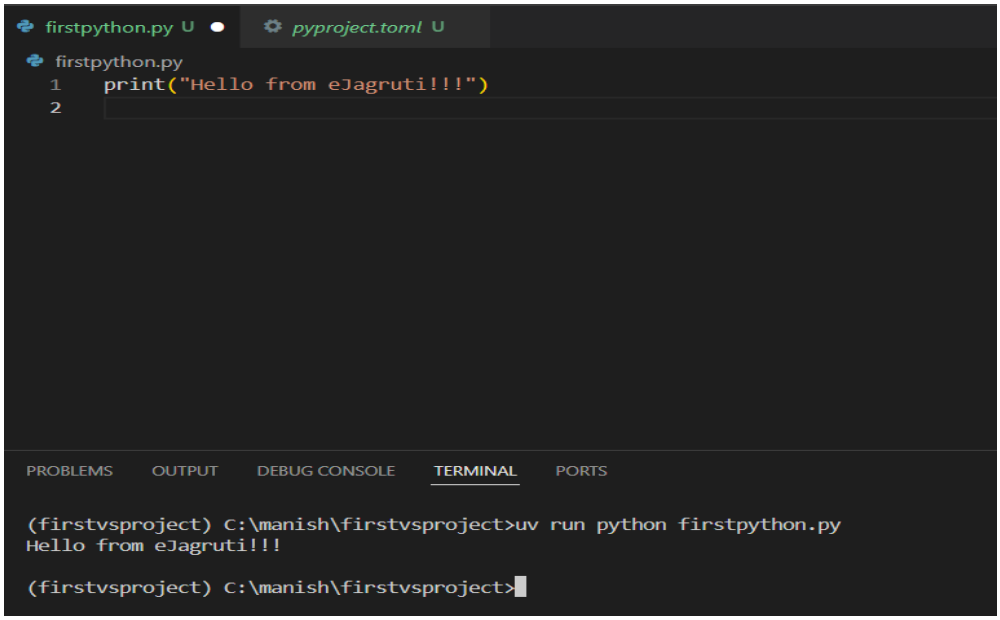
23. This will create .venv folder in your project folder



24. To set the reference for virtual environment run the command



25. To run the same code for this virtual environment



*** The End***